

Chapter 6

Computer Architecture

(part II)

Textbook and References

Textbook:

Digital Design and Computer Architecture, 2nd Edition, David M.H and Sarah L.H, Morgan Kaufmann, Elsevier, 2013
(ISBN: 978-0-12-394424-5)

Reference(s):

1. Computer Architecture and Implementation, Harvey G. Cragon, Cambridge University Press, 2000 (ISBN: 0-52-165168-9)
2. Computer Architecture: A Quantitative Approach 4th Edition, David A. Patterson and John L. Hennessy, Morgan Kaufmann Publishers, 2005. (ISBN: 0-12-370490-1)

Simulator:

Tasm (Turbo Assembler)

Link: https://en.wikipedia.org/wiki/MIPS_instruction_set

Objective(s)

- One of the objectives of this section is to make each student to familiar with ISA (Instruction Set Architecture) -Machine Language of a CPU.
- Once the students gain the knowledge of Machine Language (ISA) of a CPU, then it is possible for them to design and simulate the ISA for a given CPU size (either a 16-bit CPU or 32-bit CPU or any) by using any conventional programming language .
- Finally, the students will get the knowledge of the complete operation of a computer.

Branching

- An advantage of a computer over a calculator **is its ability to make decisions**. A computer performs different tasks depending on the input.
 - For example, ***if-else*** statements, ***switch/case*** statements, ***while*** loops, and ***for*** loops, etc all conditionally executable code depending on some test.
- To sequentially execute instructions, in a MIPS system, the **program counter (PC)** increments by 4 after each instruction (why?)
- Branch instructions modify the **PC** to skip over sections of code or to repeat previous code.
 - ***Conditional branch*** instructions perform a test and branch only if the test is TRUE.
 - ***Unconditional branch*** instructions, called *jumps*, always branch.

Conditional Branches

- The MIPS instruction set has **two conditional branch instructions**:
 1. **branch if equal (beq)** and
 2. **branch if not equal (bne)**.
- **beq** branches when the values in two registers are equal, and **bne** branches when they are not equal.
- **Code Example 6.12** illustrates the use of **beq**.
 - Note that branches are written as **beq rs, rt, imm**, where **rs** is the first source register. This order is reversed from most **I-type** instructions.

Conditional Branch - beq

Code Example 6.12 CONDITIONAL BRANCHING USING beq

MIPS Assembly Code

```
addi $s0, $0, 4      # $s0 = 0 + 4 = 4
addi $s1, $0, 1      # $s1 = 0 + 1 = 1
sll  $s1, $s1, 2      # $s1 = 1 << 2 = 4
beq  $s0, $s1, target # $s0 == $s1, so branch is taken
addi $s1, $s1, 1      # not executed
sub  $s1, $s1, $s0     # not executed

target:
add  $s1, $s1, $s0     # $s1 = 4 + 4 = 8
```

Conditional Branch- beq

- When the program in **Code Example 6.12** reaches the **branch if equal (beq)** instruction, the value in \$s0 is equal to the value in \$s1, so the **branch is taken.**
- That is, the next instruction executed is the **add** instruction just after the label called **target.**
- **The two instructions directly after the branch and before the label are not executed.**

Conditional Branch- bneq

- **Code Example 6.13** shows an example using the **branch if not equal (bne)** instruction.
- In this case, the branch is **not taken** because **\$s0** is equal to **\$s1**, and the code continues to execute directly after the **bne** instruction. All instructions in this code snippet are executed.

Conditional Branch- bneq

Code Example 6.13 CONDITIONAL BRANCHING USING bne

MIPS Assembly Code

```
addi  $s0, $0, 4      # $s0 = 0 + 4 = 4
addi  $s1, $0, 1      # $s1 = 0 + 1 = 1
sll   $s1, $s1, 2      # $s1 = 1 << 2 = 4
bne   $s0, $s1, target # $s0 == $s1, so branch is not taken
addi  $s1, $s1, 1      # $s1 = 4 + 1 = 5
sub   $s1, $s1, $s0     # $s1 = 5 - 4 = 1

target:
add   $s1, $s1, $s0     # $s1 = 1 + 4 = 5
```

Unconditional Branch- jump

- A program can unconditionally branch, or jump, using the **three types of jump instructions**:
 1. **jump (j)**,
 2. **jump and link (jal)**, and
 3. **jump register (jr)**.
- **Jump (j)** jumps directly to the instruction at the specified label.
- **Jump and link (jal)** is similar to **j** instruction but is used by functions to save a return address.
- **Jump register (jr)** jumps to the address which is held in a register.
 - **Code Example 6.14** shows the use of the **jump** instruction (**j**).

Unconditional Branch- jump

Code Example 6.14 UNCONDITIONAL BRANCHING USING j

MIPS Assembly Code

```
addi $s0, $0, 4    # $s0 = 4
addi $s1, $0, 1    # $s1 = 1
j     target       # jump to target
addi $s1, $s1, 1    # not executed
sub  $s1, $s1, $s0  # not executed

target:
add  $s1, $s1, $s0  # $s1 = 1 + 4 = 5
```

Unconditional Branch- jump register

- After the **j target instruction**, the program in **Code Example 6.14** unconditionally continues executing the add instruction at the label **target**.
- All of the instructions between the jump and the label are skipped.
- **Code Example 6.15** shows the use of the **jump register instruction (jr)**.
- Instruction addresses are given to the left of each instruction.
- **jr \$s0** jumps to the address held in \$s0, 0x00002010.

Unconditional Branch- jump register

Code Example 6.15 UNCONDITIONAL BRANCHING USING jr

MIPS Assembly Code

```
0x00002000  addi  $s0, $0, 0x2010  # $s0 = 0x2010
0x00002004  jr     $s0             # jump to 0x00002010
0x00002008  addi  $s1, $0, 1        # not executed
0x0000200c  sra   $s1, $s1, 2       # not executed
0x00002010  lw     $s3, 44($s1)  # executed after jr instruction
```

Conditional Statements

- ***if*, *if/else*, and *switch/case*** statements are **conditional statements** commonly used in **high-level languages**.
 - They each **conditionally execute** a block of code consisting of one or more statements.

If Statements

- An ***if*** statement executes a block of code, the ***if*** block, only when a condition is met.
- **Code Example 6.16** shows how to translate an ***if*** statement into **MIPS assembly code**.

If Statements

Code Example 6.16 if STATEMENT

High-Level Code

```
if (i == j)
    f = g + h;

f = f - i;
```

MIPS Assembly Code

```
# $s0 = f, $s1 = g, $s2 = h, $s3 = i, $s4 = j
    bne $s3, $s4, L1    # if i != j, skip if block
    add $s0, $s1, $s2    # if block: f = g + h
L1:
    sub $s0, $s0, $s3    # f = f - i
```

If Statements

- The assembly code for the *if* statement tests the opposite condition of the one in the high-level code.
- In **Code Example 6.16**, the high-level code tests for $i == j$, and the assembly code tests for $i \neq j$.
- The ***bne*** instruction branches (*skips the if block*) when $i \neq j$. Otherwise, $i == j$, **the branch is not taken**, and the *if* block is executed as desired.

if/else Statements

- **if/else** statements execute one of two blocks of code depending on a condition.
 - When the condition in the **if** statement is met, the **if block** is executed. Otherwise, the **else block** is executed.
 - **Code Example 6.17** shows an example **if/else** statement.
- Like **if statements**, **if/else** assembly code tests the opposite condition of the one in the high-level code.
 - For example, in **Code Example 6.17**, the high-level code tests for $i == j$. The assembly code tests for the opposite condition ($i \neq j$).
 - If that opposite condition is TRUE, **bne** skips the **if block** and executes the **else block**.
 - Otherwise, the **if block** executes and finishes with a
 - **jump instruction (j)** to jump past the **else block**.

if/else Statements

Code Example 6.17 if/else STATEMENT

High-Level Code

```
if (i == j)
    f = g + h;
else
    f = f - i;
```

MIPS Assembly Code

```
# $s0 = f, $s1 = g, $s2 = h, $s3 = i, $s4 = j
    bne $s3, $s4, else    # if i != j, branch to else
    add $s0, $s1, $s2     # if block: f = g + h
    j    L2               # skip past the else block
else:
    sub $s0, $s0, $s3     # else block: f = f - i
L2:
```

switch/case Statements

- **switch/case** statements execute one of several blocks of code depending on the conditions.
 - If no conditions are met, the **default block** is executed.
- A **case statement** is equivalent to a series of nested **if/else** statements.
- **Code Example 6.18** shows two high-level code snippets with the same functionality: they calculate the fee for an ATM (automatic teller machine) withdrawal of \$20, \$50, or \$100, as defined by amount.
- The MIPS assembly implementation is the same for both high-level code snippets.

switch/case Statements

High-Level Code

```
switch (amount) {  
    case 20:  fee = 2; break;  
  
    case 50:  fee = 3; break;
```

MIPS Assembly Code

```
# $s0 = amount, $s1 = fee  
case20:  
    addi $t0, $0, 20      # $t0 = 20  
    bne  $s0, $t0, case50 # amount == 20? if not,  
                          # skip to case50  
    addi $s1, $0, 2      # if so, fee = 2  
    j     done           # and break out of case  
  
case50:  
    addi $t0, $0, 50      # $t0 = 50  
    bne  $s0, $t0, case100 # amount == 50? if not,  
                          # skip to case100  
    addi $s1, $0, 3      # if so, fee = 3  
    j     done           # and break out of case
```

switch/case Statements

```
case 100: fee = 5; break;
```

```
default: fee = 0;
```

```
}
```

```
// equivalent function using  
if/else statements
```

```
if      (amount == 20) fee = 2;  
else if (amount == 50) fee = 3;  
else if (amount == 100) fee = 5;  
else           fee = 0;
```

```
case100:
```

```
    addi $t0, $0, 100      # $t0 = 100  
    bne  $s0, $t0, default # amount == 100? if not,  
                           # skip to default  
    addi $s1, $0, 5        # if so, fee = 5  
    j     done             # and break out of case
```

```
default:
```

```
    add  $s1, $0, $0       # fee = 0
```

```
done:
```

Loops

- **Loops** repeatedly execute a block of code depending on a condition.
- **For** loops and **while** loops are common loop constructs used by high-level languages.
- This section shows how to translate them into MIPS assembly language.

While Loops

- **while** loops repeatedly execute a block of code until a condition is not met.
 - The **while** loop in **Code Example 6.19** determines the value of x such that $2^x = 128$. It executes seven times, until $pow = 128$. If the opposite condition is TRUE, the **while** loop is finished.

while Loop

Code Example 6.19 while LOOP

High-Level Code

```
int pow = 1;
int x = 0;

while (pow != 128)
{
    pow = pow * 2;
    x = x + 1;
}
```

MIPS Assembly Code

```
# $s0 = pow, $s1 = x
addi $s0, $0, 1      # pow = 1
addi $s1, $0, 0      # x = 0

addi $t0, $0, 128    # t0 = 128 for comparison
while:
    beq $s0, $t0, done # if pow == 128, exit while loop
    sll $s0, $s0, 1    # pow = pow * 2
    addi $s1, $s1, 1   # x = x + 1
    j    while
done:
```

while Loop

- In Code **Example 6.19**, the while loop compares *pow* to 128 and exits the loop if it is *equal*. Otherwise it doubles *pow* (using a **left shift**), increments *x*, and jumps back to the start of the **while loop**.

for Loop

- **for loops**, like **while loops**, repeatedly execute a block of code until a condition is not met.
- However, **for loops** add support for a **loop control variable (LCV)**, which typically keeps track of the number of loop executions.
- A general format of the **for loop** is
for (initialization of LCV; condition; LCV update)
{ statement }
- The *initialization of LCV* of executes before the **for loop** begins.
- The condition is tested at the beginning of each loop. If the condition is not met, the loop exits. The loop operation executes at the end of each loop.

for Loop

- **Code Example 6.20** adds the numbers from 0 to 9. The loop variable, in this case *i*, is **LV** and is initialized to 0 and is incremented at the end of each loop iteration.
- At the beginning of each iteration, the for loop executes only
- when *i* is not equal to 10. Otherwise, the loop is finished.
- In this case, the **for loop** executes 10 times. ***for loops*** can be implemented using a ***while loop***, but the ***for loop*** is often convenient.

for Loop

Code Example 6.20 for LOOP

High-Level Code

```
int sum = 0;

for (i = 0; i != 10; i = i + 1) {
    sum = sum + i ;
}

// equivalent to the
following while loop

int sum = 0;
int i = 0;
while (i != 10) {
    sum = sum + i;
    i = i + 1;
}
```

MIPS Assembly Code

```
# $s0 = i, $s1 = sum
add    $s1, $0, $0      # sum = 0
addi   $s0, $0, 0       # i = 0
addi   $t0, $0, 10      # $t0 = 10

for:
    beq    $s0, $t0, done # if i == 10, branch to done
    add    $s1, $s1, $s0  # sum = sum + i
    addi   $s0, $s0, 1    # increment i
    j      for

done:
```

Arrays

- **Arrays** are useful for accessing large amounts of similar data.
- An **array** is organized as sequential data addresses in memory.
- Each **array element** is identified by a number called its **index**.
- The number of elements in the array is called the **size of the array**.
- This section shows how to access array elements in memory.

Array Indexing

- **Figure 6.20** shows an **array** of five integers stored in memory.
- The **index** ranges from 0 to 4. In this case, the array is stored in a processor's **main memory** starting at **base address** 0x10007000.

```
int[] array = new int[5]; // in Java
```

- The **base address** gives the address of the first array element, **array[0]**.
- **Code Example 6.21** multiplies the first two elements in array by 8 and stores them back into the array.

Array Indexing

Address	Data
0x10007010	array[4]
0x1000700C	array[3]
0x10007008	array[2]
0x10007004	array[1]
0x10007000	array[0]

Main Memory

Figure 6.20 Five-entry array with base address of 0x10007000

Array Indexing

Code Example 6.21 ACCESSING ARRAYS

High-Level Code

```
int array[5];
```

```
array[0] = array[0] * 8;
```

```
array[1] = array[1] * 8;
```

MIPS Assembly Code

```
# $s0 = base address of array
```

```
lui $s0, 0x1000      # $s0 = 0x10000000
```

```
ori $s0, $s0, 0x7000 # $s0 = 0x10007000
```

```
lw  $t1, 0($s0)      # $t1 = array[0]
```

```
sll $t1, $t1, 3      # $t1 = $t1 << 3 = $t1 * 8
```

```
sw  $t1, 0($s0)      # array[0] = $t1
```

```
lw  $t1, 4($s0)      # $t1 = array[1]
```

```
sll $t1, $t1, 3      # $t1 = $t1 << 3 = $t1 * 8
```

```
sw  $t1, 4($s0)      # array[1] = $t1
```


Array Indexing

- The **first step** in accessing an **array element** is to **load** the **base address** of the array into a register.
- **Code Example 6.21** loads the base address into \$s0. Recall that the **load upper immediate (lui)** and **or immediate (ori)** instructions can be used to load a 32-bit constant into a register.
 - **Code Example 6.21** also illustrates why **lw** takes a **base address** and an **offset**. The base address points to the start of the array.
 - The **offset** can be used to access subsequent elements of the array.
 - For example, **array[1]** is stored at memory address 0x10007004 (one word or four bytes after **array[0]**), so it is accessed at an offset of 4 past the base address.

Array Indexing

- **Code Example 6.22** uses a **for loop** to multiply by 8 all of the elements of a **1000-element array** stored at a **base address** of 0x23B8F000.
 - **Figure 6.21** shows the 1000-element array in memory.
 - The index into the **array** is now a variable (*i*) rather than a constant, so we cannot take advantage of the immediate offset in **lw**.
- Instead, we compute the address of the *i*th element and store it in register \$t0.
- Remember that each array element is a word but that memory is **byte addressed**, so the offset from the base address is *i**4.
 - **Shifting left by 2** is a convenient way to **multiply by 4** in MIPS assembly language. This example readily extends to an array of any size.

Array Indexing

Code Example 6.22 ACCESSING ARRAYS USING A for LOOP

High-Level Code

```
int i;  
int array[1000];  
  
for (i = 0; i < 1000; i = i + 1)  
  
    array[i] = array[i] * 8;
```

MIPS Assembly Code

```
# $s0 = array base address, $s1 = i  
# initialization code  
lui  $s0, 0x23B8          # $s0 = 0x23B80000  
ori  $s0, $s0, 0xF000     # $s0 = 0x23B8F000  
addi $s1, $0, 0           # i = 0  
addi $t2, $0, 1000        # $t2 = 1000  
  
loop:  
    slt  $t0, $s1, $t2     # i < 1000?  
    beq  $t0, $0, done     # if not, then done  
    sll  $t0, $s1, 2        # $t0 = i*4 (byte offset)  
    add  $t0, $t0, $s0      # address of array[i]  
    lw   $t1, 0($t0)        # $t1 = array[i]  
    sll  $t1, $t1, 3        # $t1 = array[i] * 8  
    sw   $t1, 0($t0)        # array[i] = array[i] * 8  
    addi $s1, $s1, 1        # i = i + 1  
    j    loop              # repeat  
done:
```

Array Indexing

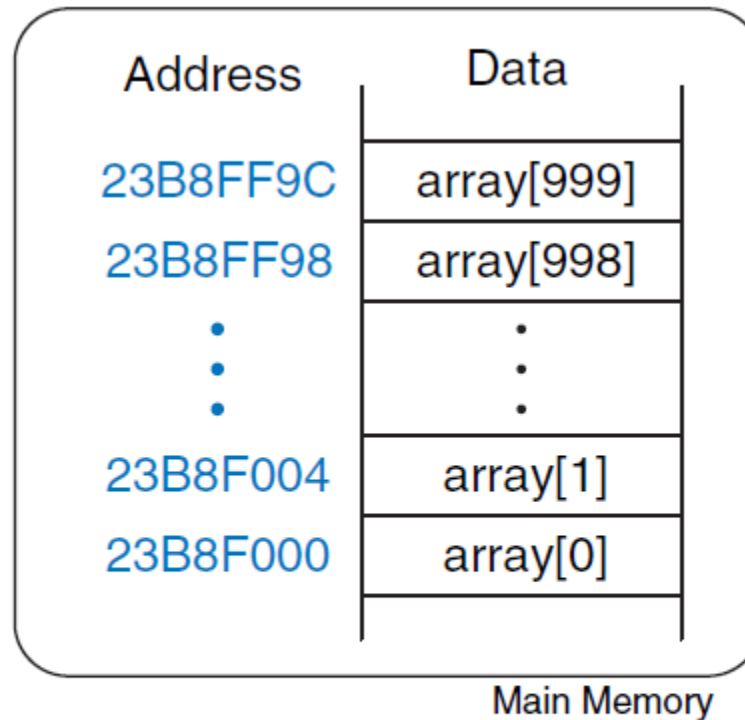


Figure 6.21 Memory holding `array[1000]` starting at base address `0x23B8F000`

Bytes and Characters

- Numbers in the range $[-128, 127]$ can be stored in a single **byte** rather than an entire word.
 - Because there are much fewer than 256 characters on an English language keyboard, English characters are often represented by **bytes**.
 - The **C language** uses the **type char** to represent a byte or character.
- In 1963, the American Standards Association published the **American Standard Code for Information Interchange (ASCII)**, which assigns each text character a unique byte value.
 - **Table 6.2** shows these character encodings for printable characters. The **ASCII** values are given in hexadecimal. Lower-case and upper-case letters differ by 0x20 (32).

ASCII values

Table 6.2 ASCII encodings

#	Char	#	Char	#	Char	#	Char	#	Char	#	Char
20	space	30	0	40	@	50	P	60	`	70	p
21	!	31	1	41	A	51	Q	61	a	71	q
22	"	32	2	42	B	52	R	62	b	72	r
23	#	33	3	43	C	53	S	63	c	73	s
24	\$	34	4	44	D	54	T	64	d	74	t
25	%	35	5	45	E	55	U	65	e	75	u
26	&	36	6	46	F	56	V	66	f	76	v
27	'	37	7	47	G	57	W	67	g	77	w
28	(	38	8	48	H	58	X	68	h	78	x
29	)	39	9	49	I	59	Y	69	i	79	y

ASCII values

Table 6.2 ASCII encodings

#	Char	#	Char	#	Char	#	Char	#	Char	#	Char
2A	*	3A	:	4A	J	5A	Z	6A	j	7A	z
2B	+	3B	;	4B	K	5B	[	6B	k	7B	{
2C	,	3C	<	4C	L	5C	\	6C	l	7C	
2D	-	3D	=	4D	M	5D	]	6D	m	7D	}
2E	.	3E	>	4E	N	5E	^	6E	n	7E	~
2F	/	3F	?	4F	O	5F	_	6F	o		

Function Calls

- High-level languages often use **functions** (also called **procedures**) *to reuse frequently accessed code and to make a program more modular and readable.*
 - **Functions** have **inputs**, called **arguments**, and an **output**, called the **return value**.
 - Functions should calculate the **return value** and cause no other unintended *side effects*.
- MIPS uses the **jump and link instruction (jal)** to **call** a function and the **jump register instruction (jr)** to **return** from a function.
 - **Code Example 6.23** shows the *main* function calling the *simple* function, the *main* function is the **caller**, and *simple* is the **callee** (called function). The *simple* function is called with no input arguments and generates no return value; it simply returns to the caller.

Function Calls

Code Example 6.23 simple FUNCTION CALL

High-Level Code

```
int main() {  
    simple();  
    ...  
}  
// void means the function returns no value  
void simple() {  
    return;  
}
```

MIPS Assembly Code

```
0x00400200 main:    jal simple    # call function  
0x00400204          ...  
  
0x00401020 simple: jr $ra        # return
```

Function Calls

- **Jump and link (jal)** and **jump register (jr \$ra)** are the two essential instructions needed for a function call.
- **jal** performs **two operations**:
 - it stores the address of the next instruction (the instruction after jal) in the return address register (\$ra), and
 - it jumps to the target instruction.
- In **Code Example 6.23**, the *main function* calls the *simple function* by executing the **jump and link (jal)** instruction.
 - **jal** jumps to the *simple label* and stores 0x00400204 in \$ra. The simple function returns immediately by executing the instruction **jr \$ra**, jumping to the instruction address held in \$ra.
 - The main function then continues executing at this address (0x00400204).

Function Calls

Input Arguments and Return Values

- The simple function in **Code Example 6.23** is not very useful, because it receives no input from the calling function (main) and returns no output.
- By MIPS convention, functions use \$a0–\$a3 for input arguments and \$v0–\$v1 for the return value.
- In **Code Example 6.24**, the *function diffofsums* is called with four arguments and returns one result.
 - According to MIPS convention, the **calling function**, *main*, places the function arguments from left to right into the input registers, \$a0–\$a3.
 - The **called function**, *diffofsums*, stores the return value in the return register, \$v0.

Function Calls

Code Example 6.24 FUNCTION CALL WITH ARGUMENTS AND RETURN VALUES

High-Level Code

```
int main()
{
    int y;

    . . .

    y = diffofsums(2, 3, 4, 5);

    . . .
}

int diffofsums(int f, int g, int h, int i)
{
    int result;
    result = (f + g) - (h + i);
    return result; }
```

MIPS Assembly Code

```
# $s0 = y
main:
    . . .
    addi $a0, $0, 2    # argument 0 = 2
    addi $a1, $0, 3    # argument 1 = 3
    addi $a2, $0, 4    # argument 2 = 4
    addi $a3, $0, 5    # argument 3 = 5
    jal  diffofsums    # call function
    add  $s0, $v0, $0   # y = returned value
    . . .
# $s0 = result
diffofsums:
    add $t0, $a0, $a1   # $t0 = f + g
    add $t1, $a2, $a3   # $t1 = h + i
    sub $s0, $t0, $t1   # result = (f + g) - (h + i)
    add $v0, $s0, $0    # put return value in $v0
    jr  $ra             # return to caller
```

The Stack

- The **stack** is a memory segment that is used to save **local variables** within a function.
 - The **stack** expands (uses more memory) as the processor needs more scratch space and contracts (uses less memory) when the processor no longer needs the variables stored there.
- The **stack** is a **last-in-first-out (LIFO) queue**. Like a stack of dishes, the last item **pushed** onto the stack (the top dish) is the first one that can be pulled (**popped**) off.
 - Each **function** may allocate stack space to store **local variables** but must **de-allocate** it before returning.
 - **The top of the stack, is the most recently allocated space.**
 - Whereas a **stack** of dishes grows up in space, the MIPS stack grows down in memory. The stack expands to lower memory addresses when a program needs more scratch space.

The Stack

- **Figure 6.24** shows a picture of the **stack**.
- The **stack pointer, \$sp, is a special MIPS register that points to the top of the stack.**
- A **pointer** points to (gives the address of) data.
- For example, in **Figure 6.24(a)** the stack pointer, **\$sp**, holds the address value **0x7FFFFFFC** and points to the data value **0x12345678**.
- **\$sp** points to the **top of the stack**, the lowest accessible memory on the stack.
- Thus, in **Figure 6.24(a)**, the stack cannot access memory below memory word 0x7FFFFFFC.

The Stack

- The **stack pointer (\$sp)** starts at a high memory address and decrements to expand as needed.
- **Figure 6.24(b)** shows the stack expanding to allow two more data words of temporary storage.
- To do so, **\$sp** decrements by 8 to become 0x7FFFFFFF4. Two additional data words, 0xAABBCCDD and 0x11223344, are temporarily stored on the stack.
- **One of the important uses of the stack is to save and restore registers that are used by a function.**
- In particular, it should not modify any registers besides the one containing the return value **\$v0**.

The Stack

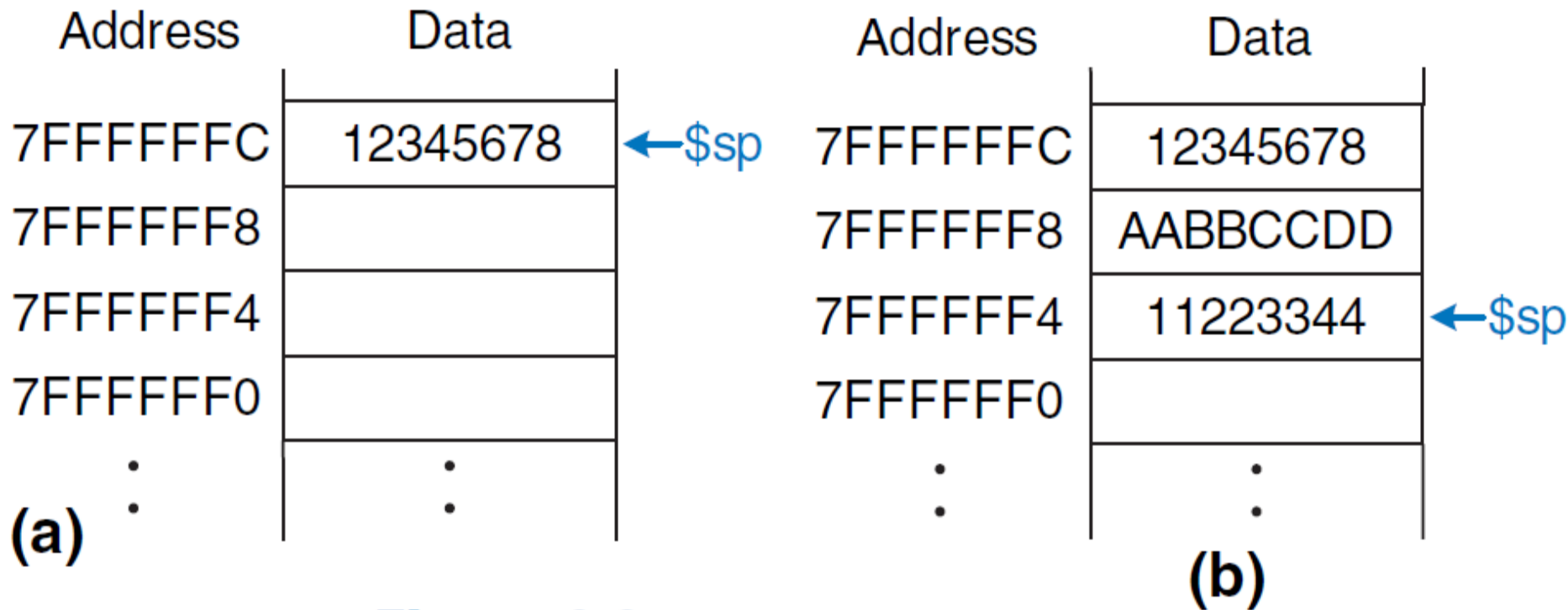


Figure 6.24 The stack

The Stack

- **Code Example 6.25** shows an improved version of *diffofsums* that saves and restores \$t0, \$t1, and \$s0. The new lines are indicated in blue.
- **Figure 6.25** shows the **stack** before, during, and after a call to the *diffofsums* function from **Code Example 6.25**. *diffofsums* makes room for three words on the stack by decrementing the stack pointer **\$sp** by 12.
 - It then stores the current values of \$s0, \$t0, and \$t1 in the newly allocated space. It executes the rest of the function, changing the values in these three registers.
 - At the end of the function, *diffofsums* restores the values of \$s0, \$t0, and \$t1 from the stack, *deallocates* its stack space, and returns. When the function returns, \$v0 holds the result, but there are no other side effects: \$s0, \$t0, \$t1, and \$sp have the same values as they did before the function call.

Function Saving Registers on the Stack: Code Example 6.25.

MIPS Assembly Code

```
# $s0 = result
diffofsums:
    addi $sp, $sp, -12 # make space on stack to store three registers
    sw   $s0, 8($sp)   # save $s0 on stack
    sw   $t0, 4($sp)   # save $t0 on stack
    sw   $t1, 0($sp)   # save $t1 on stack
    add  $t0, $a0, $a1 # $t0 = f + g
    add  $t1, $a2, $a3 # $t1 = h + i
    sub  $s0, $t0, $t1 # result = (f + g) - (h + i)
    add  $v0, $s0, $0   # put return value in $v0
    lw   $t1, 0($sp)   # restore $t1 from stack
    lw   $t0, 4($sp)   # restore $t0 from stack
    lw   $s0, 8($sp)   # restore $s0 from stack
    addi $sp, $sp, 12  # deallocate stack space
    jr   $ra           # return to caller
```

The Stack

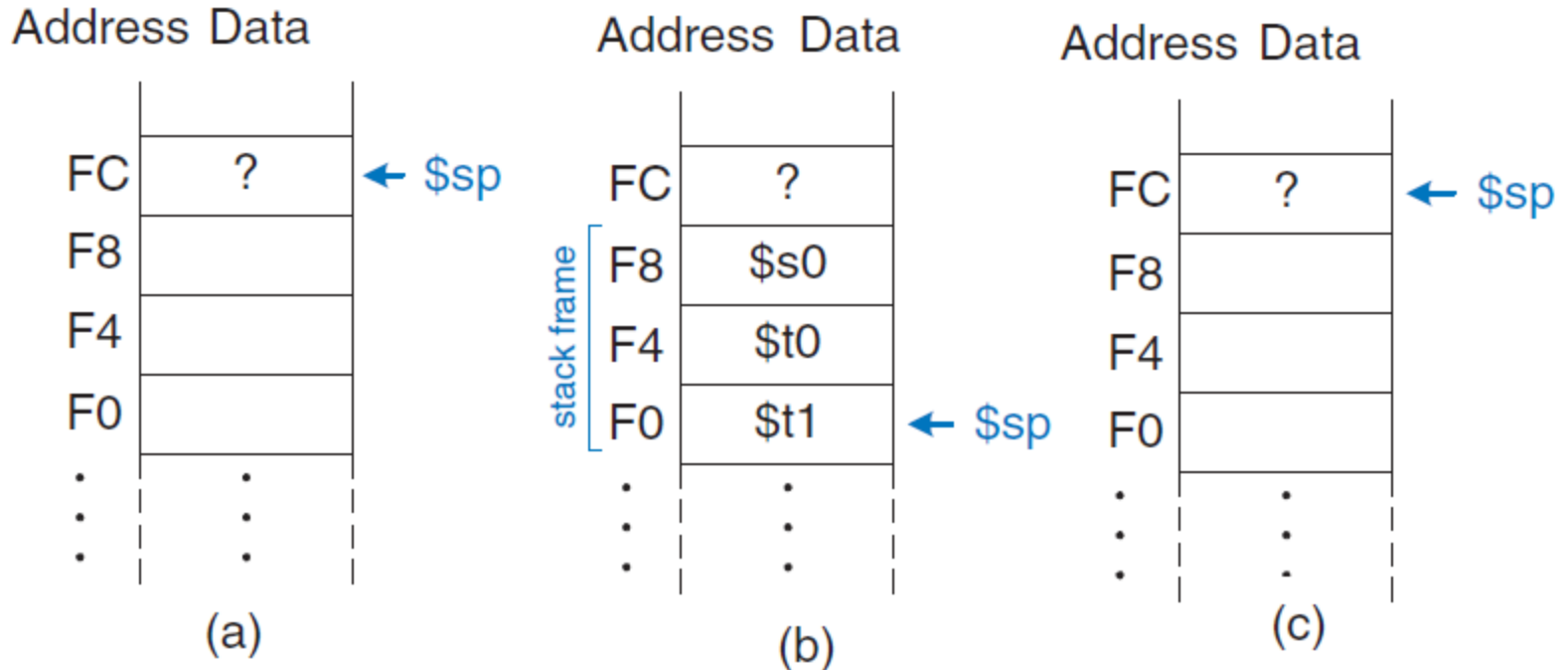


Figure 6.25 The stack (a) before, (b) during, and (c) after `diffofsums` function call

Addressing Modes

- MIPS instruction set uses **five addressing modes**:
 1. **register-only**,
 2. **immediate**,
 3. **base**,
 4. **PC-relative**, and
 5. **pseudo-direct**.
- The **first three modes** (*register-only*, *immediate*, and *base* addressing) define modes of reading and writing operands.
- The **last two modes** (*PC-relative* and *pseudo-direct* addressing) define modes of writing the program counter (PC).

Addressing Modes

Register-Only Addressing

- **Register-only addressing** uses registers for all source and destination operands.
- All R-type instructions use register-only addressing.

Immediate Addressing

- Immediate addressing uses the **16-bit immediate** along with registers as operands.
- The 16-bit immediate will be converted into a 32-bit value
- Some I-type instructions, such as add immediate (addi) and load upper immediate (lui), use immediate addressing.

Addressing Modes

Base Addressing

- Memory access instructions, such as **load word (lw)** and **store word (sw)**, use **base addressing**.
- The **effective address** of the memory operand is found by adding the **base address** in register **rs** to the **sign-extended 16-bit offset** found in the immediate field.

PC-Relative Addressing

- **Conditional branch** instructions use PC-relative addressing to specify the new value of the PC if **the branch is taken**.
- The signed offset in the immediate field is added to the PC to obtain the **new PC**; hence, the **branch destination address** is said to be relative to the **current PC**.

Addressing Modes

Addressing mode	Example instruction	Meaning	When used
Register	Add R4,R3	$\text{Regs}[R4] \leftarrow \text{Regs}[R4] + \text{Regs}[R3]$	When a value is in a register.
Immediate	Add R4,#3	$\text{Regs}[R4] \leftarrow \text{Regs}[R4] + 3$	For constants.
Displacement	Add R4,100(R1)	$\text{Regs}[R4] \leftarrow \text{Regs}[R4] + \text{Mem}[100 + \text{Regs}[R1]]$	Accessing local variables (+ simulates register indirect, direct addressing modes).
Register indirect	Add R4,(R1)	$\text{Regs}[R4] \leftarrow \text{Regs}[R4] + \text{Mem}[\text{Regs}[R1]]$	Accessing using a pointer or a computed address.
Indexed	Add R3,(R1+R2)	$\text{Regs}[R3] \leftarrow \text{Regs}[R3] + \text{Mem}[\text{Regs}[R1] + \text{Regs}[R2]]$	Sometimes useful in array addressing: R1 = base of array; R2 = index amount.
Direct or absolute	Add R1,(1001)	$\text{Regs}[R1] \leftarrow \text{Regs}[R1] + \text{Mem}[1001]$	Sometimes useful for accessing static data; address constant may need to be large.
Memory indirect	Add R1,@(R3)	$\text{Regs}[R1] \leftarrow \text{Regs}[R1] + \text{Mem}[\text{Mem}[\text{Regs}[R3]]]$	If R3 is the address of a pointer p , then mode yields $*p$.
Autoincrement	Add R1,(R2)+	$\begin{aligned} \text{Regs}[R1] &\leftarrow \text{Regs}[R1] + \text{Mem}[\text{Regs}[R2]] \\ \text{Regs}[R2] &\leftarrow \text{Regs}[R2] + d \end{aligned}$	Useful for stepping through arrays within a loop. R2 points to start of array; each reference increments R2 by size of an element, d .
Autodecrement	Add R1,-(R2)	$\begin{aligned} \text{Regs}[R2] &\leftarrow \text{Regs}[R2] - d \\ \text{Regs}[R1] &\leftarrow \text{Regs}[R1] + \text{Mem}[\text{Regs}[R2]] \end{aligned}$	Same use as autoincrement. Autodecrement/-increment can also act as push/pop to implement a stack.
Scaled	Add R1,100(R2)[R3]	$\text{Regs}[R1] \leftarrow \text{Regs}[R1] + \text{Mem}[100 + \text{Regs}[R2] + \text{Regs}[R3] * d]$	Used to index arrays. May be applied to any indexed addressing mode in some computers.

MIPS Memory Map

The Memory Map

- With **32-bit addresses**, the MIPS address space spans 2^{32} bytes = 4 gigabytes (GB).
- Word addresses are divisible by 4 and range from 0 to 0xFFFFFFF0.
- **Figure 6.31** shows the MIPS memory map.
- The MIPS architecture divides the memory address space into **four segments**:
 - the text segment,
 - global data segment,
 - dynamic data segment, and
 - reserved segments.
- The following sections describe each segment.

MIPS Memory Map

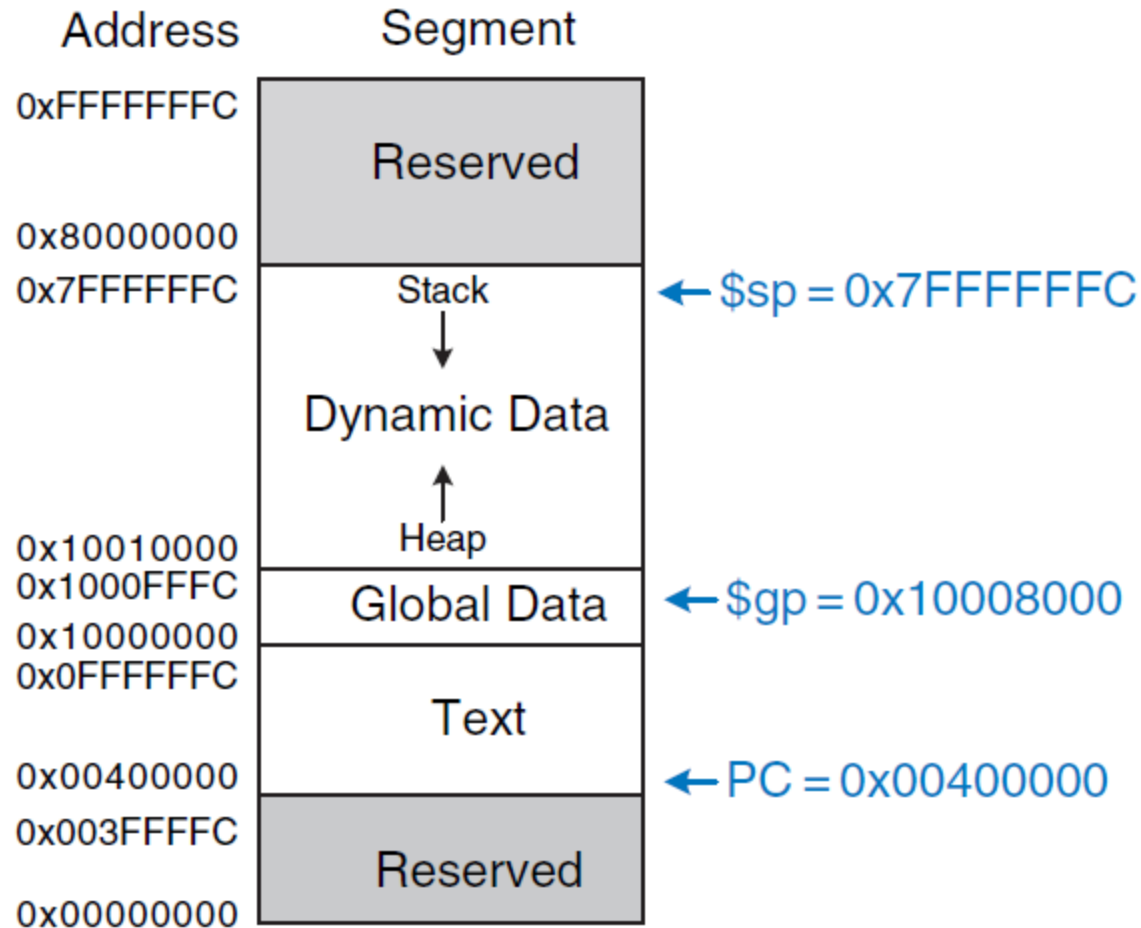


Figure 6.31 MIPS memory map

MIPS Memory Map

The Text Segment

- The ***text segment*** stores the machine language program (generated by a compiler called **macro-Instruction set**) form the user code. It is large enough to accommodate almost 256 MB of code.

The Global Data Segment

- The ***global data segment*** stores ***global variables*** that, in contrast to ***local variables***, can be seen by all functions in a program.
 - **Global variables** are defined at *start-up*, before the program begins executing.
 - These variables are declared outside the **main function** in a C program and can be accessed by any function.
 - The **global data segment** is large enough to store **64 KB** of global variables.

MIPS Memory Map

- **Global variables** are accessed using the **global pointer (\$gp)**, which is initialized to 0x100080000.
 - Unlike the **stack pointer (\$sp)**, **\$gp** does not change during program execution.
- Any global variable can be accessed with a 16-bit positive or negative **offset** from **\$gp**.
- The **offset** is known at assembly time, so the variables can be efficiently accessed using **base addressing mode** with constant offsets.

MIPS Memory Map

The Dynamic Data Segment

- The ***dynamic data segment*** holds the **stack** and the **heap**.
- The data in this segment are not known at start-up but are **dynamically allocated** and **de-allocated** throughout the execution of the program.
- **This is the largest segment of memory used by a program, spanning almost 2 GB of the address space.**
 - the **stack** is used to save and restore registers used by **functions** and to hold **local variables** such as **arrays**.
 - The **heap** stores data that is allocated by the program during runtime. In C, memory allocations are made by the ***malloc*** function; in C++ and Java, ***new*** is used to allocate memory.

MIPS Memory Map

The Reserved Segments

- The *reserved segments* of the main memory are used by the **operating system** and cannot directly be used by the program.

Translating and Starting a Program

- **Figure 6.32** shows the steps required to **translate a program from a high-level language into machine language** and to start executing that program.
 - First, the **high-level code is compiled into its assembly code form**.
 - Then **assembly code** is assembled into **machine code** in an ***object file*** form.
 - After that, the ***linker*** combines the ***machine code*** with ***object code*** with **libraries** and other **files** to produce an entire **executable program (.exe file)** and the **loader** sends it to the main memory.
 - from the memory, CPU fetches the **machine code** and executes the program.
- In practice, most compilers perform all the steps of ***compiling, assembling, and linking*** and **loading**.

Translating and Starting a Program

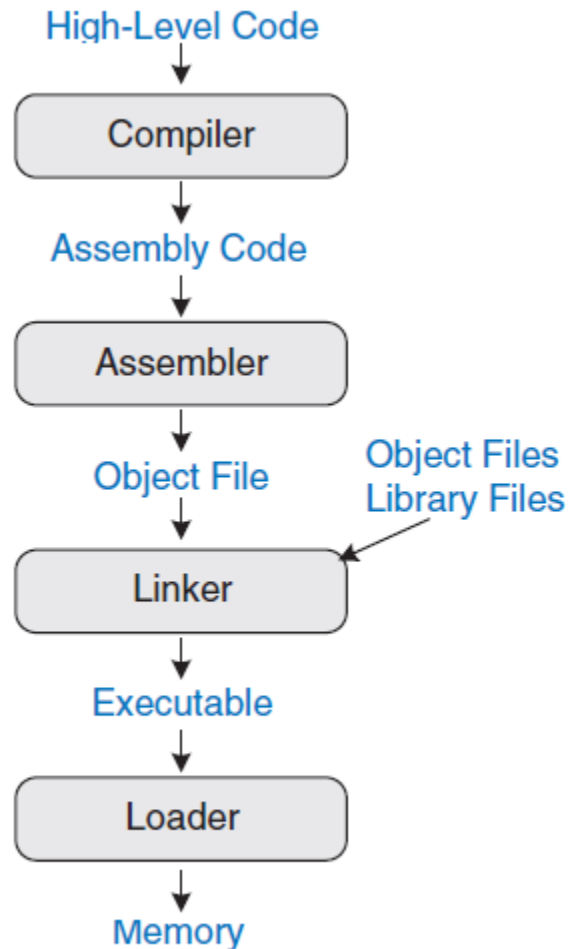


Figure 6.32 Steps for translating and starting a program

Translating and Starting a Program

Step 1: Compilation

- A **compiler** translates high-level code into **assembly language (it is equal to the instruction of the CPU called macro-instruction set)**.
- **Code Example 6.30** shows a simple high-level program with **three global variables** and **two functions**, along with the assembly code produced by a typical compiler.
- The **.data** and **.text** keywords are assembler directives that indicate where the **text and data segments memories** begin.
 - Labels are used for global variables **f**, **g**, and **y**. Their storage location will be determined by the assembler; for now, they are left as symbols in the code.

Translating and Starting a Program

Code Example 6.30 COMPILING A HIGH-LEVEL PROGRAM

High-Level Code

```
int f, g, y; // global variables
```

```
int main(void)
```

```
{  
    f = 2;  
    g = 3;  
    y = sum(f, g);  
    return y;  
}
```

```
int sum(int a, int b) {  
    return (a + b);  
}
```

MIPS Assembly Code

```
.data
```

```
f:
```

```
g:
```

```
y:
```

```
.text
```

```
main:
```

```
    addi $sp, $sp, -4    # make stack frame  
    sw   $ra, 0($sp)    # store $ra on stack  
    addi $a0, $0, 2      # $a0 = 2  
    sw   $a0, f          # f = 2  
    addi $a1, $0, 3      # $a1 = 3  
    sw   $a1, g          # g = 3  
    jal  sum             # call sum function  
    sw   $v0, y          # y = sum(f, g)  
    lw   $ra, 0($sp)     # restore $ra from stack  
    addi $sp, $sp, 4     # restore stack pointer  
    jr   $ra             # return to operating system
```

```
sum:
```

```
    add  $v0, $a0, $a1   # $v0 = a + b  
    jr   $ra             # return to caller
```

Translating and Starting a Program

Step 2: Assembling

- The **assembler** turns the **assembly language code** into an **object file** containing **machine language code**.
 - The assembler makes **two passes** through the assembly code.
- **On the first pass**, the assembler assigns instruction addresses and finds all the symbols, such as labels and global variable names.
- The code after the **first assembler pass** is shown here.

Translating and Starting a Program

- The code after the **first assembler pass** is shown here.

```
0x00400000  main: addi $sp, $sp, -4
0x00400004          sw  $ra, 0($sp)
0x00400008          addi $a0, $0, 2
0x0040000C          sw  $a0, f
0x00400010          addi $a1, $0, 3
0x00400014          sw  $a1, g
0x00400018          jal  sum
0x0040001C          sw  $v0, y
0x00400020          lw  $ra, 0($sp)
0x00400024          addi $sp, $sp, 4
0x00400028          jr   $ra
0x0040002C  sum:  add  $v0, $a0, $a1
0x00400030          jr   $ra
```

Translating and Starting a Program

- The **names** and **addresses** of the symbols are kept in a **symbol table**, as shown in **Table 6.4** for this code.
- The ***symbol addresses*** are filled in **after the first pass**, when the addresses of labels are known.
 - Global variables are assigned storage locations in the global data segment of memory, starting at memory address 0x10000000.
- **On the second pass** through the code, the assembler produces the **machine language code**.
 - Addresses for the **global variables** and labels are taken from the **symbol table**.
 - **The machine language code and symbol table are stored in the object file.**

Translating and Starting a Program

Table 6.4 Symbol table

Symbol	Address
f	0x10000000
g	0x10000004
y	0x10000008
main	0x00400000
sum	0x0040002C

Translating and Starting a Program

Step 3: Linking

- In particular, programs often call functions in **library files**; these library files almost never change.
- The job of the **linker** is to combine all of the **object files** into **one machine language file** called the **executable**.
- The **linker** relocates the data and instructions in the **object files** so that they are not all on top of each other.
 - It uses the information in the **symbol tables** to adjust the addresses of global variables and of labels that are relocated.

Translating and Starting a Program

- **Figure 6.33** shows the **executable file**. It has three sections:
 1. the **executable file header**,
 2. the **text segment**, and
 3. the **data segment**.
- The **executable file** header reports the **text size** (*code size*) and **data size** (amount of *globally declared data*).
- Both are given in units of bytes.
- The **text segment** gives the instructions in the order that they are stored in memory.

Translating and Starting a Program

Executable file header	Text Size	Data Size
	0x34 (52 bytes)	0xC (12 bytes)
Text segment	Address	Instruction
	0x00400000	0x23BDFFFC
	0x00400004	0xAFBF0000
	0x00400008	0x20040002
	0x0040000C	0xAF848000
	0x00400010	0x20050003
	0x00400014	0xAF858004
	0x00400018	0x0C10000B
	0x0040001C	0xAF828008
	0x00400020	0x8FBF0000
	0x00400024	0x23BD0004
	0x00400028	0x03E00008
	0x0040002C	0x00851020
	0x00400030	0x03E00008
Data segment	Address	Data
	0x10000000	f
	0x10000004	g
	0x10000008	y

```

addi $sp, $sp, -4
sw  $ra, 0($sp)
addi $a0, $0, 2
sw  $a0, 0x8000($gp)
addi $a1, $0, 3
sw  $a1, 0x8004($gp)
jal  0x0040002C
sw  $v0, 0x8008($gp)
lw  $ra, 0($sp)
addi $sp, $sp, -4
jr   $ra
add  $v0, $a0, $a1
jr   $ra

```

Figure 6.33 Executable

Translating and Starting a Program

Step 4: Loading

- The **operating system (OS)** loads a program by reading the **text segment of the executable file** from a storage device (usually the hard disk) into the **text segment of memory**.
- The **OS** sets **\$gp** to **0x10008000** (the middle of the **global data segment**) and **\$sp** to **0x7FFFFFFC** (the top of the **dynamic data segment**), then performs a **jal 0x00400000** to **jump** to the beginning of the program.
- **Figure 6.34** shows the memory map at the beginning of program execution.

Translating and Starting a Program

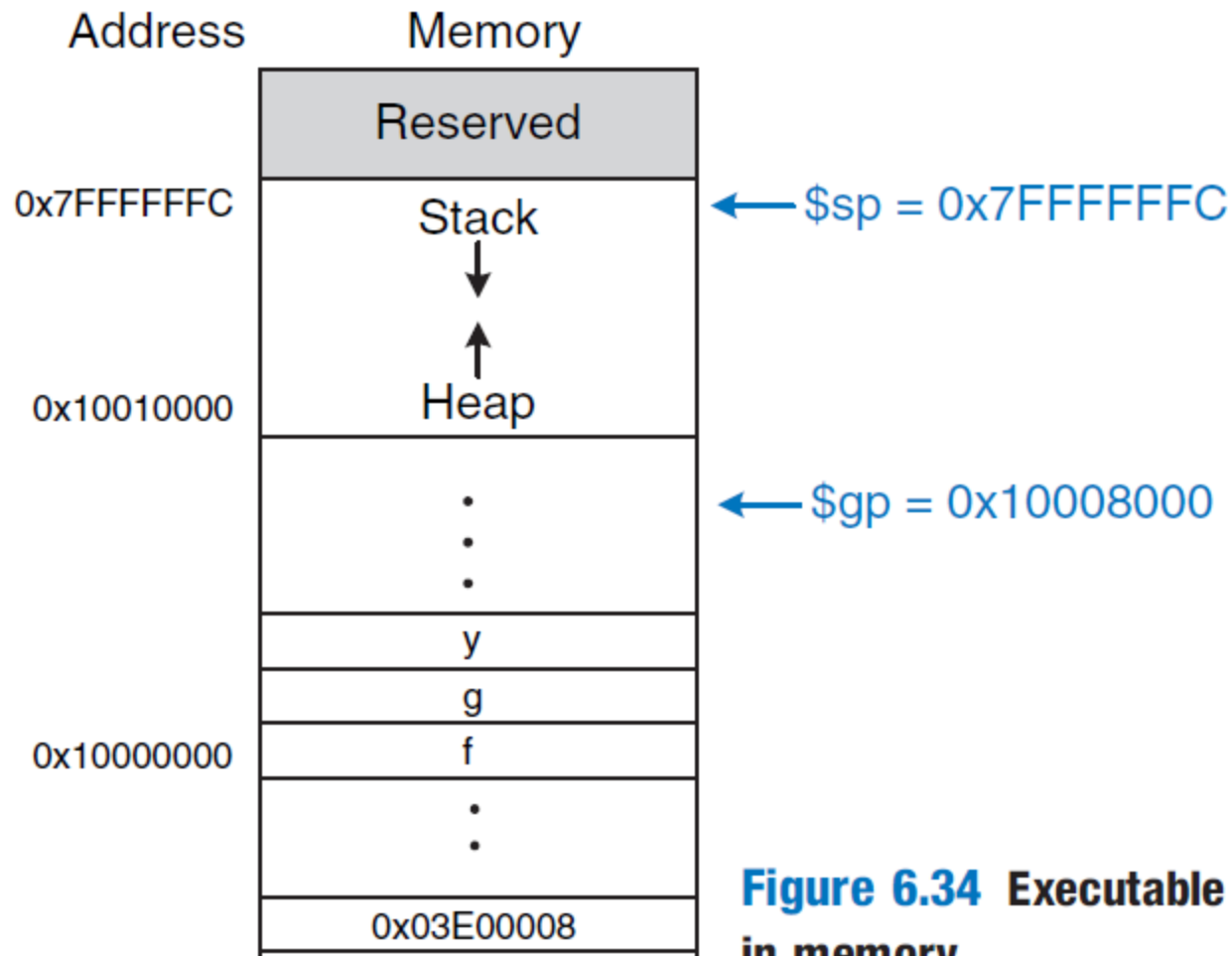


Figure 6.34 Executable loaded in memory

Translating and Starting a Program

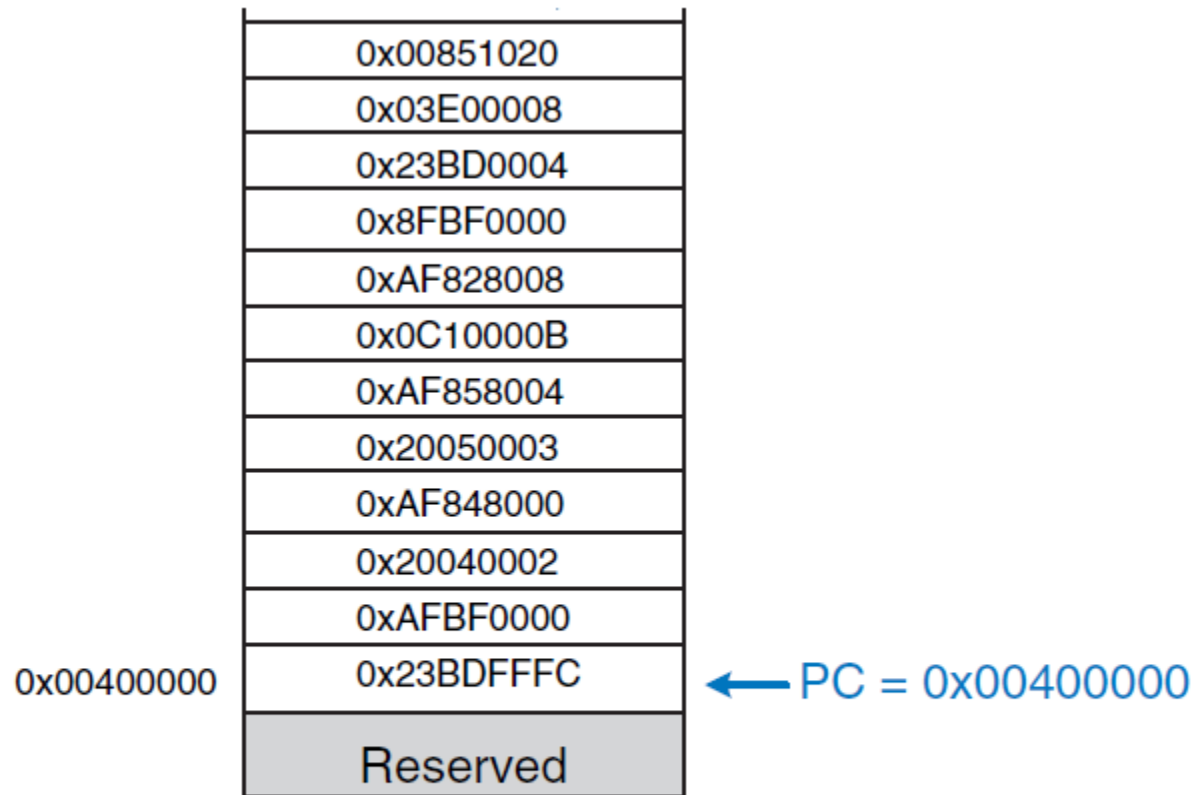


Figure 6.34 Executable loaded
in memory